

UNIVERSITÉ CÔTE D'AZUR
ÉCOLE UNIVERSITAIRE DE RECHERCHE – DIGITAL SYSTEMS FOR HUMANS

M1 INFORMATIQUE

Configuration-Space Exploration of Language-Model Pipelines for Constraint Satisfaction Problems

A Progressive Delegation Ablation Study

Student	[Your full name]
Student ID	[Your student ID]
Programme	M1 Informatique, parcours DS4H
Supervisor	[Supervisor name]
Host laboratory	[Laboratory name]
Recipient	DS4H Programme Director, EUR DS4H Jury
Project period	Spring semester 2026

Tutorship final report
Submission: 12 May 2026

Abstract

Constraint Satisfaction Problems (CSPs) underlie scheduling, planning, and resource-allocation tasks across many industries. Solving them at scale requires either rare human expertise or automated tools. This study investigates whether Large Language Models (LLMs) can drive a classical CSP solver, and how three design choices interact in such a pipeline: the prompting technique, the LLM itself, and the agentic design that connects the LLM to the solver. A grid of configurations was enumerated, and an in-depth study of the agentic-design axis was conducted under the name Progressive Delegation Ablation (PDA). The PDA compares four pipeline depths, from a single all-in-one agent (L0) to a four-agent chain (L3). Three LLMs and ten CSPLib-anchored benchmarks were used. Under the tested conditions, the configuration with the shallowest delegation depth obtained the highest success rate, and deeper configurations exhibited a monotonic decline (100 % at L0, 93.3 % at L1, 80.0 % at L2, 66.7 % at L3 on average across models). Five mechanisms are proposed to explain the decline, and a single principle is suggested to unify them: when the task includes a deterministic external checker, additional LLM agents tend to compete with that checker rather than complement it. This effect is named the *verifier-redundancy tax*.

Keywords: Constraint Satisfaction Problems; Large Language Models; multi-agent pipelines; neuro-symbolic systems; **ablation study; Choco solver; CSPLib.**

Contents

1	Introduction	3
1.1	Why combinatorial problems matter	3
1.2	Why language models are a candidate	3
1.3	Three design choices	3
1.4	Research questions	3
1.5	Contributions	3
1.6	Outline of the report	4
2	Background	4
2.1	Constraint Satisfaction Problems	4
2.2	Large Language Models	4
2.3	The three design axes	4
2.4	Self-correcting pipelines	5
2.5	Neuro-symbolic perspective	5
2.6	Related work on multi-agent LLM systems	5
3	Materials and Methods	5
3.1	The configuration-space approach	5
3.2	The agentic-design axis: Progressive Delegation Ablation	5
3.2.1	The five behaviours	5
3.2.2	The four-rung ladder	6
3.3	The prompting axis	6
3.4	The model axis	7
3.5	The self-correction loop	7
3.6	Enumeration procedure	7
3.7	Benchmark suite	7
3.8	Metrics	7
3.9	Tools and reproducibility	7

4	Results	8
4.1	The configuration grid	8
4.2	What changes along the delegation axis	8
4.3	What changes along the model axis	8
4.4	Hard-tier breakdown	9
4.5	The cost dimension	9
4.6	Verdict on the pre-registered predictions	10
5	Discussion	10
5.1	Identifying the best configuration	10
5.2	Five mechanisms behind the decline	10
5.3	The verifier-redundancy tax	11
5.4	Relation to prior literature	11
5.5	Limitations	11
6	Conclusion	11
6.1	Summary	11
6.2	Future work	11

1 Introduction

1.1 Why combinatorial problems matter

Combinatorial problems appear in industrial scheduling, project management, human-resource allocation, financial portfolio construction, and many other settings –mention fields and applications that are more appealing and more marketing–. A small input description can hide an exponential search space –what the hell does that mean !!!–. Even when a solution exists, finding one by hand is rarely feasible. Classical operations-research tools –why did you mention this ! it will overwhelm the reader !!– can solve such problems efficiently, but they require a human expert to first translate the informal description into a formal model. That expertise remains scarce.

1.2 Why language models are a candidate

Recent LLMs read natural-language descriptions well and produce structured code with reasonable reliability [1, 2]. They are therefore a plausible interface between an informal problem statement –in natural language– and a deterministic –stricter ...– solver. The role of the LLM in such a pipeline –say system and not pipeline– is restricted: it must produce a formal model, not solve the search –that is done by the solver–. The actual search is delegated to a classical solver whose behaviour is predictable. This division of labour –‘the task dispatch and delegation’ is better– matches the broader pattern of *neuro-symbolic* systems, described further in Section 2.5.

1.3 Three design choices

Building such a pipeline –a system– requires choosing three components. First, a *prompting technique* (the way the request is phrased and explained). Second, an *LLM* (the model that produces the formal output). Third, an *agentic design* (whether one agent does everything, or several agents each handle one task–the reader here doesn't have any clue what agents are and you're already throwing that word at him !!–). Each axis admits many options, and the three axes interact –form the final result–. A study that varies one axis at a time risks confounding effects; a study that varies all three simultaneously enumerates a large grid. –what does a large grid mean !!! use better explanatory words–

–i want you here to point out as a marketing point that the automatic solver is generic and not proper to a single problem and handicap against other problems–

1.4 Research questions

Three questions are pursued.

- **RQ1.** How can LLMs be used to solve CSPs –don't mention 'using a solver' yet, talk in more abstract way–?
- **RQ2.** Which combination of prompting technique, LLM, and agentic design yields the highest CSP-solving performance under the tested conditions?
- **RQ3.** Can such a pipeline correct itself autonomously when the produced model fails to compile or to satisfy the problem?

1.5 Contributions

This work introduces four contributions.

1. A configuration-space sweep –what the hell does that mean to the reader !!!– over the three design axes, with comparable metrics per cell, yielding a direct answer to **RQ2**.

2. A focused ablation –the reader doesn't know what ablation is yet !!!– of the agentic-design axis, named here the *Progressive Delegation Ablation* (PDA). Four delegation depths are compared, from a single agent (L0) to a four-agent chain (L3).
3. An empirical –use another word along with empirical– finding that contradicts a common assumption in the multi-agent-LLM literature: on a task that already provides a deterministic external checker, increasing the number of LLM agents reduces success monotonically across the tested levels.
4. A causal account –what the hell does that mean !– composed of five mechanisms, summarised under a single named principle, the *verifier-redundancy tax*.

1.6 Outline of the report

Section 2 introduces the technical background and positions the work in the neuro-symbolic and multi-agent-LLM literature. Section 3 describes the configuration-space approach and the PDA design. Section 4 presents the experimental results. Section 5 discusses the mechanisms behind the observed decline and the proposed unifying principle. Section 6 concludes and lists future directions. –refactor this, you're throwing random technical words to the reader !!–

2 Background

2.1 Constraint Satisfaction Problems

A Constraint Satisfaction Problem (CSP) is a triple $\mathcal{P} = (V, D, C)$, where $V = \{v_1, \dots, v_n\}$ is a set of variables, $D = \{D_1, \dots, D_n\}$ assigns to each variable a finite domain of admissible values, and C is a set of constraints that restrict the allowed combinations of values [10]. A solution is an assignment of one value from D_i to each v_i such that every constraint in C is satisfied.

The 4-Queens problem is a small example: V contains four queen positions, each D_i contains the four columns, and C forbids same-column, same-row, and same-diagonal placements. Classical solvers such as Choco [9] explore the search space with propagation and backtracking, and are deterministic given a fixed input. –give an illustrative drawing to demonstrate this problem and its solution–

2.2 Large Language Models

An LLM is a neural network trained to predict the next token in a sequence. Modern LLMs operate over a bounded context window, accept formatted instructions, and produce structured output when explicitly required [7, 1]. Their strengths include in-context generalisation from a few examples and the production of syntactically valid code. Their weaknesses include silent semantic errors (outputs that appear correct but encode the wrong meaning), drift over long contexts, and sensitivity to small prompt changes. These weaknesses motivate both the use of an external checker and the self-correction loop discussed in Section 2.4. –don't use technical words such that they are known to the user !! say 'generalisation from a few examples, which is called in-context', better than introducing the term 'in-context' first and overwhelming the reader. you can also put an illustrative figure here–

2.3 The three design axes

Three knobs govern a pipeline of the kind studied here. The *prompting technique* determines what information the model receives and how it is structured. Common families include in-context examples (few-shot), explicit step-by-step reasoning –directives– (Chain-of-Thought [2]), and self-reflective revision (Reflexion [3]). The *model* choice determines the underlying capability. Open and commercial models differ in size, training data, and reliability on code generation. The

agentic design determines whether one model instance performs all tasks or several instances each handle one task in a chain. These three axes are not independent: a strong model may compensate for a weak prompting technique, and a deep chain may be too expensive for a slow model.

2.4 Self-correcting pipelines

A self-correcting pipeline reads the output of a previous step, detects a failure (a compilation error, a solver rejection, a validator complaint), and produces a corrected attempt. The loop is bounded by a retry budget. This pattern provides the mechanism that answers **RQ3**: the pipeline is autonomous in the sense that, within the budget, it requires no human intervention to recover from typical errors. –that was kind of a good explanation, with no overwhelming terms–

2.5 Neuro-symbolic perspective

The pipeline –system– under study fits the broader class of *neuro-symbolic* systems, in which a neural component (the LLM) is coupled with a symbolic component (the –combinatorial–solver) [11]. In such systems, the LLM operates as the reasoning engine, and the domain to be solved is treated as *business logic* to be integrated into the LLM environment –you can do better at explaining it simpler–. A medical-diagnosis chatbot integrates clinical decision rules as business logic; a legal-advice assistant integrates statute-and-case logic; the present work integrates CSP modelling as business logic. The recurring engineering question is the same across these domains: how to couple a probabilistic reasoner with a deterministic backend so that the deterministic component remains the source of truth.

2.6 Related work on multi-agent LLM systems

Several frameworks decompose a single user request across multiple LLM agents that exchange messages. AutoGen [4] provides a generic conversational scaffolding. MetaGPT [5] simulates a software-engineering team with role specialisation. ChatDev [6] adopts a similar division of labour for end-to-end software development. These systems report performance gains over single-agent baselines on tasks such as code-writing, design, and debate. A property of those tasks deserves attention: none of them includes an automatic external checker that decides correctness. The present study evaluates the same decomposition idea on a task that *does* include such a checker, and the contrast becomes central in Section 5.

3 Materials and Methods

3.1 The configuration-space approach

Each candidate system is identified by a triple $c = (\pi, m, \alpha)$, where π is a prompting technique, m is an LLM, and α is an agentic design. The set $C = \Pi \times M \times A$ of all such triples forms the configuration space. The same benchmark suite was applied to each cell, the same metrics were recorded, and the cells were compared on equal footing. The best cell answers **RQ2** at the surface level; the analysis of the dominant axis answers it in depth.

3.2 The agentic-design axis: Progressive Delegation Ablation

3.2.1 The five behaviours

Five behaviours are required to convert a natural-language problem statement into a verified solution. They are constant across all configurations.

- **Formalisation** – extract variables, domains, and constraints from the description.

- **Modelling** – generate compilable Java code that expresses the formalisation in the Choco API.
- **Validation** – perform a semantic check on the model before the solver is invoked.
- **Refinement** – read the failure signal (compiler, solver, or validator) and produce a fix.
- **Explanation** – produce a narrative grounded in the solver trace.

What changes across configurations is not the set of behaviours, but how they are *delegated* across agents.

3.2.2 The four-rung ladder

The ladder defines four agentic designs, summarised in Table 1. At each successive rung, one behaviour is detached from the generalist agent and assigned to a new specialist. The architecture of each rung is illustrated in Figure 1.

Table 1: The four-rung delegation ladder. F = Formalisation, M = Modelling, V = Validation, R = Refinement, E = Explanation.

Level	Agents	F	M	V	R	E
L0 (Monolith)	1	Monolith	Monolith	Monolith	Monolith	Monolith
L1 (Refiner split)	2	Monolith	Monolith	Monolith	Refiner	Monolith
L2 (Validator split)	3	Monolith	Monolith	Validator	Refiner	Monolith
L3 (Formaliser split)	4	Formaliser	Modeller	Validator	Refiner	Modeller

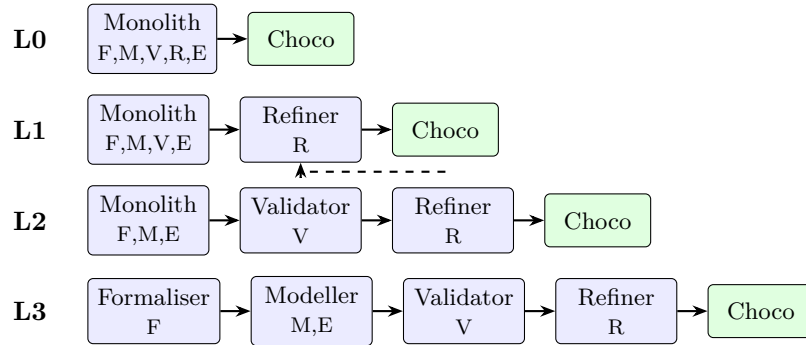


Figure 1: The four-rung delegation ladder. The same five behaviours (F, M, V, R, E) are present at every level. The Refiner is invoked when a downstream component (compiler, validator, or solver) reports a failure; the corresponding feedback edge is shown on L1 for compactness and applies at every level.

The theoretical limit being tested can be stated plainly: dividing labour across specialists is expected to reduce the cognitive load per agent and therefore raise per-call reliability. This intuition motivates the AutoGen and MetaGPT designs cited in Section 2.6.

3.3 The prompting axis

A canonical prompting technique was attached to each newly specialised agent on the ladder. The monolith uses few-shot prompting with one to two worked examples [1]. The refiner uses a Reflexion-style prompt that reads the previous error trace before rewriting the model [3]. The validator uses an adversarial Chain-of-Thought prompt that asks for defects before concluding correctness [2]. The formaliser uses structured output through a Pydantic schema, which forces a JSON object of variables, domains, and constraints. Each technique was selected for its role rather than for absolute performance; that role-fit decision is itself a contribution of the PDA design.

3.4 The model axis

Three LLMs with deliberately different profiles were selected. Qwen 2.5-Coder-32B is a code-trained open model and was chosen as a strong baseline for code generation. Claude Haiku 4.5 is a small commercial model and was chosen as a low-latency reliability baseline. DeepSeek-R1-Distill-Llama-70B is a reasoning-oriented open model and was chosen to probe whether explicit reasoning training compensates for delegation. All three were invoked through the OpenRouter API to ensure identical request semantics.

3.5 The self-correction loop

A bounded retry loop is present at every level. When the Choco backend reports a compilation error, a runtime exception, or no solution within the time budget, the refining role is invoked with the failure trace as context, and a new attempt is generated. The loop terminates upon success or after `MAX_REFINEMENT_ITERATIONS` attempts (set to three for all experiments). This loop provides the empirical answer to **RQ3**; its convergence behaviour is reported in Section 4.5.

3.6 Enumeration procedure

The configurations were enumerated by a triple-nested loop, reproduced below. The total number of runs is $|A| \times |M| \times |B|$, where A is the set of delegation levels, M is the set of models, and B is the benchmark suite.

```
for level in {L0, L1, L2, L3}:
    pipeline = build_workflow(level)
    for model in {Qwen, Haiku, DeepSeek}:
        for problem in benchmarks:
            run = pipeline.solve(problem, model)
            record_metrics(run)
```

3.7 Benchmark suite

Ten CSPs were drawn from CSPLib [8], including 4-Queens and 8-Queens (prob054), Magic Square (prob019), SEND + MORE = MONEY (prob023), Graph Colouring (prob004), Latin Square (prob003), and Sudoku (prob028). The remaining instances cover scheduling, knapsack, and a cryptarithmic variant. The suite is partitioned into three difficulty tiers (easy, medium, hard), with the hard tier containing four instances. Citing the CSPLib problem identifiers provides external validity without requiring new benchmark design.

3.8 Metrics

Each metric is tied to one or more research questions. Success rate (Choco produced a valid solution) addresses **RQ1** and **RQ2**. First-shot success rate (success at iteration zero) and mean refinement iterations address **RQ3**. Compilation-failure rate (Maven exit code) addresses **RQ1**. Wall time and solver statistics (nodes, backtracks, propagations) inform the cost dimension of **RQ2**. A regex-based explanation score is recorded but treated with caution (see Section 5.5).

3.9 Tools and reproducibility

The classical backend is Choco Solver [9] (Java, invoked through a Maven bridge). The agentic graphs were implemented in LangGraph. LLM calls were routed through OpenRouter. LangSmith was used for run tracing. The Choco solver is deterministic with respect to its input; given an identical generated Java source, the same answer and the same search statistics are produced.

4 Results

The results are presented along the axes of the configuration space. The headline metric is the success rate per cell. Cost-side metrics are reported separately. All numbers below are computed over the ten-problem suite of Section 3.7.

4.1 The configuration grid

The success rate of each cell of the (level, model) grid is reported in Table 2. The L0 row is uniform at 100 %, and the L3 row varies between 50 % and 90 % depending on the model. The grid identifies (L0, any model) and (L0–L1, Qwen) as the highest-performing cells under the tested conditions.

Table 2: Success rate per configuration cell. Rows: delegation levels. Columns: LLMs. Values are percentages over the ten-problem suite.

	Qwen 2.5-Coder-32B	Haiku 4.5	DeepSeek-R1 70B	Mean
L0	100	100	100	100.0
L1	100	100	80	93.3
L2	90	50	100	80.0
L3	90	60	50	66.7

4.2 What changes along the delegation axis

The dominant pattern is along the delegation axis. The mean success rate across the three models decreases monotonically as the pipeline deepens: 100.0 % → 93.3 % → 80.0 % → 66.7 %. The trend is presented in Figure 2. The drop is not noise: each step strictly lowers the mean. Two of the three models also decrease individually (Qwen at L2–L3; Haiku at L2–L3); DeepSeek alternates but ends below its L0 score.

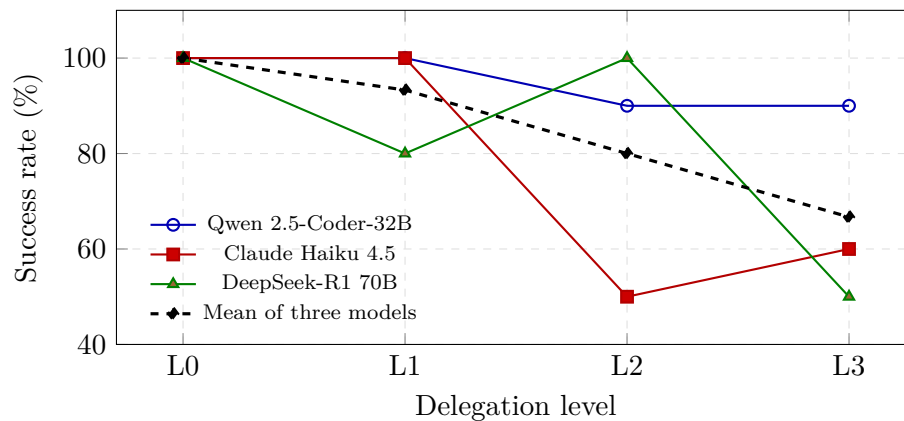


Figure 2: Success rate against delegation level. The mean across the three models declines monotonically as the pipeline deepens.

4.3 What changes along the model axis

The model axis induces three different profiles. Qwen 2.5-Coder-32B remains stable across all four levels, with only one ten-point drop between L1 and L2. Haiku 4.5 starts among the strongest at L0 and L1, then falls sharply at L2 (50 %). DeepSeek-R1 70B is irregular and the only model whose L2 value exceeds its L1 value. The implication is that no single ranking of the three models holds across the four levels: strength on the monolithic baseline does not predict strength under deeper delegation.

4.4 Hard-tier breakdown

The easy and medium tiers were solved at near 100 % across all cells; the entire signal lives in the four hard problems. Figure 3 reports the count of hard problems solved per model and per level (out of four), and the total per level (out of twelve). The total drops from 12/12 at L0 to 5/12 at L3. The four problems responsible for the collapse are Sudoku, SEND + MORE = MONEY, the scheduling instance, and the knapsack instance.

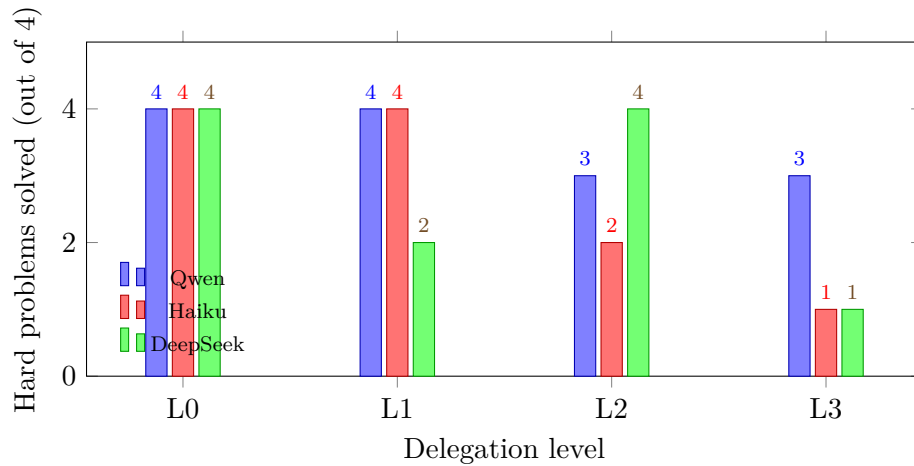


Figure 3: Hard-tier success per model and level (out of four hard problems per model). The total across the three models drops from 12/12 at L0 to 5/12 at L3.

4.5 The cost dimension

Three cost-side metrics deepen the picture. The compilation-failure rate rises from 0 % at L0 to approximately 27 % at L3. The mean number of refinement iterations approximately triples between L0 and L3. The mean wall time grows from 47 seconds at L0 to 788 seconds at L3, a factor of roughly sixteen. The wall-time mean at L3 is, however, distorted by a single outlier: one DeepSeek run on SEND + MORE = MONEY reported a 21 318-second elapsed time, traced back to a hung HTTP request rather than to model reasoning. With that single run removed, the L3 mean wall time falls substantially, although the qualitative ordering of the four levels is preserved. The compilation-failure trajectory of Haiku is reported separately in Figure 4; its non-monotonic shape is discussed in Section 5.2.

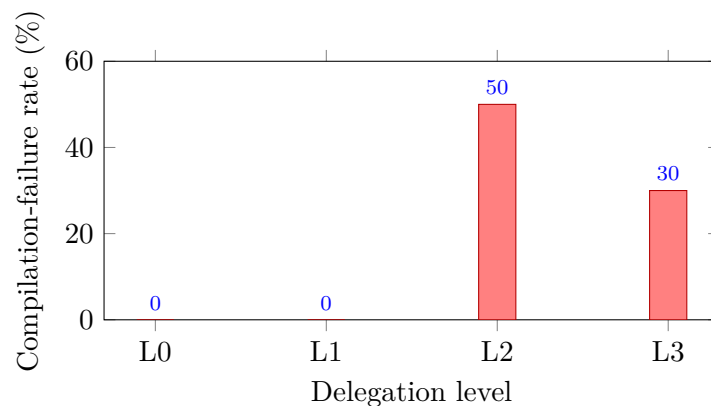


Figure 4: Claude Haiku 4.5: compilation-failure rate per delegation level. The jump at L2 coincides with the introduction of the validator-refiner loop.

4.6 Verdict on the pre-registered predictions

Four predictions were registered before measurement (**RQ1/RQ2** oriented): L1 should improve on L0; L2 should improve on L1 with fewer refinement iterations; L3 should improve on L2 on complex problems. Table 3 reports each prediction against the observed direction. All four predictions were refuted.

Table 3: Pre-registered predictions and observed outcomes.

Rung	Prediction	Observation
L1 vs. L0	Mean success up	Mean success down (100 $\rightarrow$ 93.3)
L2 vs. L1	Mean success up; iterations down	Both refuted
L3 vs. L2	Mean success up on hard tier	Hard tier 9/12 $\rightarrow$ 5/12

5 Discussion

5.1 Identifying the best configuration

Under the tested conditions, the configuration that scores highest is (L0, any model). The result is robust in the sense that the L0 row is uniform at 100 % on the ten-problem suite. This finding answers **RQ2** at the surface level. The deeper question is why deeper delegation, which prior multi-agent literature treats as beneficial, fails to improve over the monolithic baseline. The remainder of this section addresses that question.

5.2 Five mechanisms behind the decline

Five mechanisms are proposed. Each is tied to a specific datum from Section 4.

M1 – Multiplicative composition of errors. A pipeline composed of k stages, each correct with probability p , achieves a joint correctness no higher than p^k . With $p = 0.95$ and $k = 4$, the upper bound is approximately 0.81. This single mechanism is sufficient to account for the gap between L0 and L3.

M2 – Redundancy with the external checker. Choco is a deterministic and exhaustive checker: either the generated model compiles and yields a valid solution, or it does not. Inserting a probabilistic LLM critic on top of a deterministic checker adds noise without adding information. The contribution of the critic is therefore bounded above by zero in expectation, and is negative whenever the critic produces a false positive on correct code.

M3 – Critic-induced regression. A prompt that asks a model to identify defects rarely produces an empty answer. The validator therefore tends to flag correct code, and the refiner, trusting the flag, modifies it. Figure 4 provides the strongest evidence: Haiku’s compilation-failure rate jumps from 0 % at L1 to 50 % at L2 *after* the validator is added. The compilation errors at L2 were therefore introduced by the loop itself rather than produced by the original generator.

M4 – Cost-blind retry budget. The retry budget was fixed to three at every level. At L0, one retry costs one LLM call; at L3, one retry costs approximately four LLM calls, since the entire chain is re-traversed. The same nominal budget therefore consumes different amounts of compute, and the calibrated optimum at the shallow level becomes too small for genuinely hard cases and disproportionately expensive for stuck ones at the deeper level.

M5 – Model-dependent orchestration robustness. The ranking of the three models is not preserved across levels. Haiku leads at L0 and L1, then collapses at L2; DeepSeek is the only model whose L2 value exceeds its L1 value. The implication is that monolithic benchmarks do not predict pipeline behaviour, and that orchestration robustness is itself a property that should be measured separately.

5.3 The verifier-redundancy tax

The five mechanisms can be unified under one principle. Multi-agent decomposition helps when it *substitutes* for a missing external checker, as is the case for essays, design tasks, or open-ended dialogue. Multi-agent decomposition hurts when it *competes* with an external checker that is already exact and deterministic, as is the case here. The size of the penalty observed on the present benchmark is between seven and thirteen percentage points per added agent on the mean success rate. This principle is named the *verifier-redundancy tax*, and it is the second contribution restated in Section 1.5.

5.4 Relation to prior literature

The results do not contradict the existing multi-agent literature. AutoGen [4], MetaGPT [5], and ChatDev [6] evaluate decomposition on tasks that do not include a deterministic external checker, and report gains. The present study indicates that those gains do not transfer to tasks in which such a checker already exists. The observation therefore restricts, rather than refutes, the domain of validity of the prior claims.

5.5 Limitations

Five limitations should be acknowledged before further claims are drawn. First, the hard-tier sample contains four problems per model, giving twelve cells per level; the trend is monotone, but the per-cell deltas are within one standard error and a larger suite would be required to refine the effect-size estimate. Second, the Choco statistics parser produced suspiciously constant node and backtrack counts across problems of different size, so search-effort comparisons across configurations were not possible. Third, the regex-based explanation metric returned zero on all runs because the underlying solution dictionary remained empty; the metric is structurally undefined under the current logging pipeline, and a stronger explanation-faithfulness judge is required. Fourth, per-call token counts were not persisted; the cost decoupling argument associated with L3 cannot therefore be quantified. Fifth, only sequential decomposition was evaluated; debate, voting, and tree-structured decompositions remain to be tested before the verifier-redundancy tax can be generalised beyond chains.

6 Conclusion

6.1 Summary

This work introduces a configuration-space sweep over three design axes (prompting technique, LLM, agentic design) for the LLM-driven solving of Constraint Satisfaction Problems through a deterministic classical solver. The agentic-design axis is studied in depth through the Progressive Delegation Ablation, which compares four pipeline depths from a single all-in-one agent to a four-agent chain. Under the tested conditions, the shallowest configuration obtains the highest success rate, and deeper configurations exhibit a monotonic decline of the mean success rate from 100 % at L0 to 66.7 % at L3, driven almost entirely by the four hard-tier problems. Five mechanisms are proposed to explain the decline, and a single principle, the *verifier-redundancy tax*, is suggested to unify them. The contribution is methodological as much as empirical: the principle clarifies when multi-agent decomposition is expected to help and when it is expected to hurt.

6.2 Future work

Several directions follow from the limitations of Section 5.5. The order in which behaviours are delegated could be reversed, to test whether the formaliser-first split produces a different error

profile. The validator could be allowed to abstain rather than being forced to criticise, which would directly test mechanism M3. The retry budget could be made cost-aware, scaling with the per-iteration call count of the level. The solver-statistics parser should be fixed before any further claim is drawn on search effort. The hard tier should be expanded to at least twelve problems per model to achieve adequate statistical power. Finally, non-sequential decompositions (debate, voting, tree-structured) should be evaluated against the verifier-redundancy tax to test whether the principle generalises.

References

- [1] T. B. Brown et al., “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [2] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.
- [3] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [4] Q. Wu et al., “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” arXiv preprint arXiv:2308.08155, 2023.
- [5] S. Hong et al., “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] C. Qian et al., “ChatDev: Communicative agents for software development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [7] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [8] I. P. Gent and T. Walsh, “CSPLib: A benchmark library for constraints,” in *Principles and Practice of Constraint Programming (CP)*, 1999, pp. 480–481.
- [9] C. Prud’homme, J.-G. Fages, and X. Lorca, *Choco Solver Documentation*, Tasc – LS2N, IMT Atlantique, 2022. [Online]. Available: <https://choco-solver.org>
- [10] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
- [11] A. d’Avila Garcez and L. C. Lamb, “Neurosymbolic AI: The 3rd wave,” *Artificial Intelligence Review*, vol. 56, 2023, pp. 12387–12406.
- [12] LangChain Inc., *LangGraph Documentation*. [Online]. Available: <https://langchain-ai.github.io/langgraph/>